

김민준

인증의 실패 조건과 AI 연동 흐름을 설계하는 개발자

화면부터 API·배포까지 연결하며,
변경의 이유를 코드와 검증 기록으로 남깁니다.

낮선 요구사항을 작은 문제로 나누고 실패 조건을 먼저 정의합니다. PR 리뷰와 API 명세로 결정의 이유를 공유하고, 팀원이 다음 작업을 이어갈 수 있는 구조를 지향합니다.

대표 프로젝트

여기도 / Backend · 인증·회원 도메인

동시 재발급과 여러 기기를 고려한 인증 흐름

토큰 비교·교체 성공과 실패를 단위 테스트로 검증하고, 현재 세션 로그아웃과 탈퇴 시 전체 세션 정리를 구분했습니다.

상세 사례 02-04쪽

다시찾음 / Full-stack · FE·BE·배포

모바일 화면부터 AI 연동 API·배포까지

당시 초기 JavaScript 빌드 크기를 약 540KB에서 265.54KB로 줄이고, 분석 결과 연결과 만료 경계를 검증했습니다.

상세 사례 05-07쪽

추가 프로젝트 · 역량

바비튼든 · Whatshu · Crossy HUFS / 08쪽

기술 적용 경험 · 활동 이력 · 자격 / 09쪽

연락처와 자료

이메일 · <mailto:minjun0123@naver.com>

GitHub · github.com/nuj1min

웹 포트폴리오 · nuj1min.github.io

한국외국어대학교 글로벌캠퍼스 · 컴퓨터공학부

여기도

Backend · 인증·회원 도메인 / 2026.07부터 개발

지역의 숨은 명소와 문화행사, 소상공인 공간을 발견하고 여행 코스와 기록을 공유하는 지역 기반 여행 플랫폼입니다.

직접 맡은 역할

이메일·소셜 로그인부터 토큰 재발급, 프로필 관리, 비밀번호 재설정과 회원 탈퇴까지 인증·회원 도메인을 담당했습니다.

인증 생명주기 이메일 인증·회원가입·로그인, 카카오·네이버 소셜 로그인, JWT 필터, 토큰 재발급·로그아웃을 구현했습니다.

회원 정보와 상태 프로필 조회·수정과 이미지 처리, 비밀번호 재설정, 회원 탈퇴 및 탈퇴 회원 인증 차단 정책을 구현했습니다.

프론트엔드 연동 Provider별 전달값과 필수값을 구분하고, 프로필 선택값·지역 ID·Swagger 계약을 화면 요구 사항에 맞췄습니다.

여행 코스·문화 콘텐츠·소상공인 기능은 서비스 전체 범위입니다. 제 기여는 인증·회원 도메인과 프론트엔드 인증 계약을 중심으로 정리했습니다.

인증 구조의 변경

01 변경 전 · 토큰 조회 → 비교 → 별도 저장

02 변경 후 · Lua에서 비교 + 교체를 원자적으로 실행

03 확장 · userId + sessionId별 Refresh Token 관리

변경 결과

- Refresh Token 비교와 교체를 하나의 원자적 작업으로 변경했습니다.
- 세션별 저장으로 기기 간 로그인 상태를 분리하고 로그아웃·탈퇴의 정리 범위를 명확히 했습니다.
- 탈퇴 회원의 소셜 로그인 토큰 발급 경로를 차단했습니다.

기술 · Java / Spring Boot / Spring Security / JPA / MySQL / Redis / JUnit 5

백엔드 저장소 · github.com/yeongido/backend

동시 요청과 세션의 경계

인증 상태가 바뀌는 순간의 실패 조건을 다뤘습니다.

CASE 01 / 토큰 검증과 교체 사이의 경쟁 조건 줄이기

문제 기존 토큰을 읽어 비교한 뒤 새 값을 따로 저장하면, 두 재발급 요청이 모두 같은 기존 토큰 검증을 통과할 수 있었습니다.

변경 Redis Lua Script 안에서 기존 값 비교와 새 토큰 저장을 하나의 원자적 작업으로 묶었습니다. 비교·교체에 실패한 요청은 기존 AUTH4013 오류로 처리했습니다.

판단 애플리케이션의 검증과 저장 사이에 다른 요청이 끼어들 수 없도록, 두 동작을 저장소에서 함께 실행했습니다.

검증 PR #216에 원자적 교체 성공·실패 단위 테스트 통과가 기록돼 있습니다. 대규모 동시 요청 부하검증 결과를 의미하지는 않습니다.

[원자적 교체 · PR #216 · github.com/yeogido/backend/pull/216](https://github.com/yeogido/backend/pull/216)

CASE 02 / 기기별 로그인 상태와 로그아웃 범위 분리

문제 사용자별 Refresh Token 하나만 저장하면 다른 기기의 로그인이 기존 값을 덮어쓸 수 있었습니다. 로그아웃과 탈퇴가 정리할 세션 범위도 달랐습니다.

변경 로그인 시 sessionId를 발급하고 Redis 저장 키를 userId + sessionId 기준으로 변경했습니다. 로그아웃은 현재 세션, 탈퇴는 해당 사용자의 모든 Refresh Token 세션을 정리하도록 구분했습니다.

판단 여러 기기의 로그인 상태를 독립적으로 관리하면서 기존 Lua 원자적 교체도 각 세션에서 유지하도록 했습니다.

검증 PR #251에 세션별 저장·로그아웃 정책, 테스트 수정 및 동작 확인 기록이 있습니다. Refresh Token 삭제를 이미 발급한 Access Token의 즉시 폐기로 표현하지 않았습니다.

[멀티 디바이스 세션 · PR #251 · github.com/yeogido/backend/pull/251](https://github.com/yeogido/backend/pull/251)

탈퇴 상태를 인증 정책에 반영

로그인 방식이 달라도 회원 상태 검증은 일관되게 적용합니다.

CASE 03 / 탈퇴 회원이 다시 인증되는 경로 차단

문제 소셜 계정을 조회한 뒤 연결된 회원의 상태 확인 없이 토큰을 발급하는 경로가 있었습니다.

변경 이메일·소셜 로그인뿐 아니라 회원가입·이메일 인증, 비밀번호 찾기·재설정에도 탈퇴 상태 검증을 반영했습니다. 탈퇴 회원의 토큰 발급을 차단하고 USER4101 응답으로 구분했습니다.

판단 개별 로그인 방식이 달라도 회원 상태에 대한 인증 정책은 일관되게 적용되도록 했습니다.

검증 PR #301에 누락 경로 수정과 API 테스트·동작 확인 기록이 있습니다.

[탈퇴 회원 인증 정책 · PR #301 · github.com/yeogido/backend/pull/301](#)

협업 / Provider별 로그인 계약

카카오는 authorizationCode와 redirectUri를 받아 백엔드에서 토큰을 교환하고, 네이버는 프론트엔드에서 받은 accessToken을 사용하는 흐름으로 구분했습니다. Provider별 필수값과 프로필 성별 미선택 값 NONE, Swagger 계약을 함께 맞췄습니다.

PR #246의 API 테스트 완료 여부는 확인되지 않아 테스트 성과로 인용하지 않았습니다.

검증 범위

검증 내용은 해당 PR에 남은 당시 기록을 기준으로 작성했습니다. 처리속도·장애 감소율 등 측정되지 않은 운영 성과는 포함하지 않았습니다.

관련 코드와 변경 기록

[백엔드 저장소 · github.com/yeogido/backend](#)

[Provider별 소셜 로그인 계약 · PR #246 · github.com/yeogido/backend/pull/246](#)

[비밀번호 재설정 · PR #238 · github.com/yeogido/backend/pull/238](#)

[프로필 이미지 수정 · PR #310 · github.com/yeogido/backend/pull/310](#)

다시찾음

Full-stack · FE·BE·배포 / 2026.08-09 개발 기록 기준

분실물의 사진·특징·시간·장소를 한 번 등록해 경찰청 공개 습득물 중 확인할 가치가 있는 후보를 찾도록 돕는 모바일 웹 프로젝트입니다.

직접 맡은 역할

React 프론트엔드와 Spring Boot 백엔드 개발부터 배포 환경 구성까지 담당했습니다. 화면 입력, API 계약, 인증·저장, 배포 경로를 연결했습니다.

모바일 웹 로그인·회원가입, 3단계 수색카드 작성과 관리, 분석 결과 조회·재조회, 후보·피드백·알림 화면을 구현했습니다.

API와 데이터 인증·회원, 수색카드 생명주기, S3 이미지 소유권, AI 분석 요청·응답 검증·저장, 공통 응답·예외 처리를 구현했습니다.

배포 경로 Vercel의 API rewrite와 CloudFront HTTPS 연결, EC2 Docker Compose, GitHub Actions·OIDC·SSM 배포 구성을 담당했습니다.

AI 모델 실행과 DATA 수집 영역은 별도 협업 영역입니다. 후보·알림 화면 구현과 추천·발송 서버의 완성 여부를 구분하며, 모델 정확도나 분실물 회수 성과는 주장하지 않습니다.

분석 결과를 확인한 뒤 저장하는 요청 흐름

- 01 직접 구현 · 화면 입력 → BE 인증·이미지 소유권 검증
- 02 외부 AI 영역 · 내부 HTTP 계약으로 모델 분석 요청
- 03 직접 구현 · 응답 검증·저장 → 사용자 확인 → 수색카드 생성

변경 결과

- FE·BE 입력 제약을 맞추고 장소 정보와 좌표의 불일치를 방지하도록 보완했습니다.
- 초기 JavaScript 빌드 크기를 약 51% 줄였습니다. 측정 기준은 FE #58의 당시 산출물입니다.
- AI 연동의 오류·저장 계약을 분리하고 재분석 결과의 연결 제약과 수색 만료 경계를 검증했습니다.

기술 · React / TypeScript / Spring Boot / MySQL / S3 / Docker / CloudFront / GitHub Actions

[프론트엔드 · github.com/dasi-find/FE](https://github.com/dasi-find/FE)

[백엔드 · github.com/dasi-find/BE](https://github.com/dasi-find/BE)

입력의 정확성과 초기 로딩

화면과 서버의 계약을 맞추고, 첫 화면에 필요한 코드부터 불러옵니다.

CASE 01 / 사용자가 입력한 장소와 저장 좌표 맞추기

문제 지도에서 장소를 고른 후 장소명만 수정하면 기존 좌표가 남아, 화면에 보이는 장소와 실제 전송 좌표가 달라질 수 있었습니다.

변경 장소명·주소를 수정하면 좌표를 초기화하고 지도에서 다시 선택하도록 했습니다. 특징 필수 여부, 문자열 길이, 미래 날짜, 사진 제한도 백엔드 입력 제약과 대조해 생성·수정 화면에 함께 반영했습니다.

판단 입력 화면과 저장 계약을 함께 다루는 역할을 활용해, 사용자가 확인한 정보와 백엔드에 전달되는 값이 일치하도록 보완했습니다.

검증 FE #60·#62에 입력·좌표 정합성 변경이 기록돼 있습니다. #62 당시 FE 전체 88개 테스트와 빌드·정적 검사 통과 기록을 확인했습니다.

[입력·좌표 정합성 · FE #60 · github.com/dasi-find/FE/pull/60](https://github.com/dasi-find/FE/pull/60)

[수정 화면 검증 · FE #62 · github.com/dasi-find/FE/pull/62](https://github.com/dasi-find/FE/pull/62)

CASE 02 / 첫 화면에 필요한 코드부터 내려받기

문제 모든 페이지 코드가 초기 JavaScript에 포함돼, 첫 화면을 여는 데 필요하지 않은 코드까지 함께 내려받는 구조였습니다.

변경 페이지를 동적 import로 분리해 현재 주소에 필요한 화면을 불러오도록 변경했습니다. 로딩 중 공통 화면을 표시하고 초기 랜딩 렌더링 테스트를 추가했습니다.

판단 화면 단위로 코드를 나누어 첫 진입 시 받아야 하는 JavaScript의 양을 줄였습니다.

검증 FE #58 빌드 기록에서 초기 JavaScript가 약 540KB → 265.54KB로 약 51% 감소했습니다. 당시 82개 테스트와 빌드·정적 검사가 통과했습니다. 빌드 크기 감소이며 실제 로딩 시간 개선율은 아닙니다.

[화면 단위 코드 분할 · FE #58 · github.com/dasi-find/FE/pull/58](https://github.com/dasi-find/FE/pull/58)

AI 연동과 상태 경계

인증·소유권·저장은 BE에, 모델 실행은 AI 영역에 분리했습니다.

CASE 03 / AI 분석과 도메인 저장의 책임 분리

문제 AI 모델 구성과 서버 준비 상황이 달라져도 인증·저장 흐름은 독립적으로 개발할 수 있어야 했습니다. 연결 실패와 잘못된 분석 응답도 구분할 필요가 있었습니다.

변경 모델을 백엔드에서 직접 실행하는 대신 내부 분석 API를 호출했습니다. 사진이 없으면 빈 배열을 허용하고, 사진이 있으면 소유권 검증 후 유효기간이 제한된 Presigned URL을 전달했습니다. 응답 형식을 검증해 성공한 결과만 저장하고 연결·시간 초과와 응답 오류를 구분했습니다.

판단 BE는 인증·소유권·검증·저장을, AI는 모델 실행과 분석 JSON을 책임지는 계약으로 역할을 분리했습니다.

검증 BE #42 당시 백엔드 전체 139개 테스트·빌드 통과, MySQL 8.4 Flyway V4 적용과 로컬 health UP 기록이 있습니다. 이는 AI 정확도 평가나 전체 추천 기능의 운영 검증이 아닙니다.

[AI 분석 계약 · BE #42 · github.com/dasi-find/BE/pull/42](https://github.com/dasi-find/BE/pull/42)

CASE 04 / 재분석 결과 연결과 수색 만료 경계 보호

문제 다른 사용자의 분석 결과나 이미 사용된 결과가 카드에 연결되지 않아야 했고, 스케줄러가 실행되기 전에도 만료된 카드의 수정을 막아야 했습니다.

변경 분석 결과의 소유권·사용 여부와 카드 ACTIVE 상태를 검증하고 잠금 조회 후 카드·위치를 한 트랜잭션으로 갱신했습니다. 만료 작업은 ACTIVE 카드만 조건부 갱신하고, 수정·종료 요청에서도 만료 시각을 직접 검사했습니다.

판단 정기 작업 외에도 요청 시점에서 상태 경계를 확인하고, 이미 종료된 정보는 반복 실행으로 바뀌지 않도록 했습니다.

검증 BE #52에 재분석 연결·카드 및 위치 갱신·이미지 유지 테스트, #60에 시간 경계·대상 선별·종료 정보 보존·반복 실행·만료 후 요청 거절 테스트 통과 기록이 있습니다.

[재분석 기반 수정 · BE #52 · github.com/dasi-find/BE/pull/52](https://github.com/dasi-find/BE/pull/52)

[자동 만료 · BE #60 · github.com/dasi-find/BE/pull/60](https://github.com/dasi-find/BE/pull/60)

검증 수치는 각 PR 당시 기록입니다. 최신 테스트 수나 AI 정확도 평가 수로 합산하지 않았습니다.

사진 누락 시 가중치 재분배와 후보 적합도 정책은 설계 영역이며, 매칭 엔진의 구현·평가 완료와 구분합니다.

[배포 구성 · BE #14 · github.com/dasi-find/BE/pull/14](https://github.com/dasi-find/BE/pull/14)

[HTTPS·인증 쿠키 · BE #28 · github.com/dasi-find/BE/pull/28](https://github.com/dasi-find/BE/pull/28)

[CloudFront API rewrite · FE #34 · github.com/dasi-find/FE/pull/34](https://github.com/dasi-find/FE/pull/34)

다른 도메인에서 쌓은 경험

주문 흐름, 행사 출석, 게임 개발에서 맡은 역할

바비든든

2026.07부터 개발 / Backend · 메뉴·주문·관리자 인증

교내 식당에서 QR로 메뉴를 확인하고 주문·결제·픽업 현황을 연결하는 서비스입니다.

메뉴·주문 API, 관리자 JWT 인증과 판매 상태 관리를 담당했습니다. 품질·중복 옵션·최대 수량을 검증하고 PREPARING → READY → COMPLETED 상태 전이를 제한했습니다. 주문 이벤트는 트랜잭션 커밋 이후에만 발행하도록 보완했습니다.

Spring Boot / MySQL / Docker / JWT / JUnit 5

[Repository · github.com/GDGOC-babidundeun/BE](https://github.com/GDGOC-babidundeun/BE)

Whatshu

2025.09 - 2026.02 / Backend Lead

Google Form으로 분산되던 GDG 행사 출결을 세션·멤버 단위로 관리하는 모바일 출석 서비스입니다.

백엔드 리드로 Spring Boot API·인증 구조와 배포 환경을 설계했습니다. Swagger로 프론트엔드와 요청·응답 계약을 공유하고, Docker·AWS EC2·GitHub Actions 배포 환경을 구성했습니다.

Java / Spring Boot / MySQL / Docker / AWS

[Repository · github.com/GDG-whatshu/BE](https://github.com/GDG-whatshu/BE)

Crossy HUFS

2025.03 - 2025.05 / Project Lead · Unity

한국외대 글로벌캠퍼스를 배경으로 재해석한 3D 아케이드 게임입니다.

팀장으로 기획·일정·통합을 맡고 랜덤 장애물 생성 로직을 구현했습니다. 장애물 중복 배치를 방지하고 차량·통나무 생성 및 충돌 처리를 개선했습니다. 팀원 PR 통합과 빌드 안정화, 데모 일정을 관리했습니다.

C# / Unity / OOP / GitHub Projects

[Repository · github.com/nui1min/Crossy_HUFS](https://github.com/nui1min/Crossy_HUFS)

기술과 협업 경험

한국외국어대학교 글로벌캠퍼스 · 컴퓨터공학부

Java · Spring Boot · JPA · MySQL

인증·회원·메뉴·주문 API, 도메인 관계와 트랜잭션 경계, UNIQUE 충돌 및 데이터 정합성 처리

Spring Security · JWT · Redis

이메일·OAuth 인증, 역할별 인가, 세션별 Refresh Token과 TTL, Lua 원자적 교체

React · TypeScript

모바일 웹 화면, 입력 제약과 좌표 정합성, 동적 import 기반 페이지별 코드 분할

Docker · AWS · GitHub Actions

EC2·Docker Compose, OIDC·SSM 배포, CloudFront HTTPS와 Vercel API rewrite

Test · API Documentation

토큰 교체·분석 소유권·수색 만료 경계 테스트, Swagger 및 API 계약 공유

활동 이력

2022 | 컴퓨터공학부 1학년 과대표 · 보안 학회 PNP / 의견 조율과 Linux·TCP/IP 학습

2025 | 학생회 집행부장 / 행사 기획과 구성원 커뮤니케이션

2025.09 - 2026.02 | GDG 7기 Core Member · Whatshu 백엔드 리드

2026.02 - 08 | 학생회 기획부장 · UMC 10기 / 조직 운영과 Spring 백엔드 심화

2026.07부터 | 여기도·바비튼 백엔드 / 인증·회원·메뉴·주문·관리자

2026.08 - 09 기록 | 다시찾음 풀스택 / 모바일 웹·AI 분석 연동·배포

자격

리눅스마스터 2급 · 2025.08 / 네트워크관리사 2급 · 2025.07

본 문서는 2026.09.25 기준 제공된 프로젝트 자료와 PR 기록으로 작성했습니다. 개발 기록이 확인된 기간을 표기했으며, 확인되지 않은 팀 인원이나 운영 성과는 포함하지 않았습니다.